

Reinforcement Learning

Class 7

Policy gradients

Mostly based on van Hasselt and D. Silver (UCL) lectures from 2021 and 2015; chapter 13 from Sutton & Barto's book

Context

- We have covered the core RL methods in the last classes, including the framework of MDPs, how to reach optimal policies when knowing MDPs and how to handle unknown MDPs to reach optimal policies with TD, MC and Q-learning
- In the previous class, we looked at **approximation methods** that allow to address more realistic scenarios with many states and continuous cases, but always focusing on value (or state-value) functions
- In this session, we will add to our repertoire algorithms with **policy gradient algorithms**, where we will address **directly the optimization of policies** (also through function approximation) that can select actions without having access to value functions

Policy-Based Reinforcement Learning

- In the last class, we approximated the value or action-value function using parameters $\mathbf{w}$

$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$

$$\text{or } \hat{q}(s, a, \mathbf{w}) \approx q_{\pi}(s, a)$$

- A policy was generated directly from the value function, e.g. using ϵ -greedy
- In this class, we will directly **parameterise** the policy

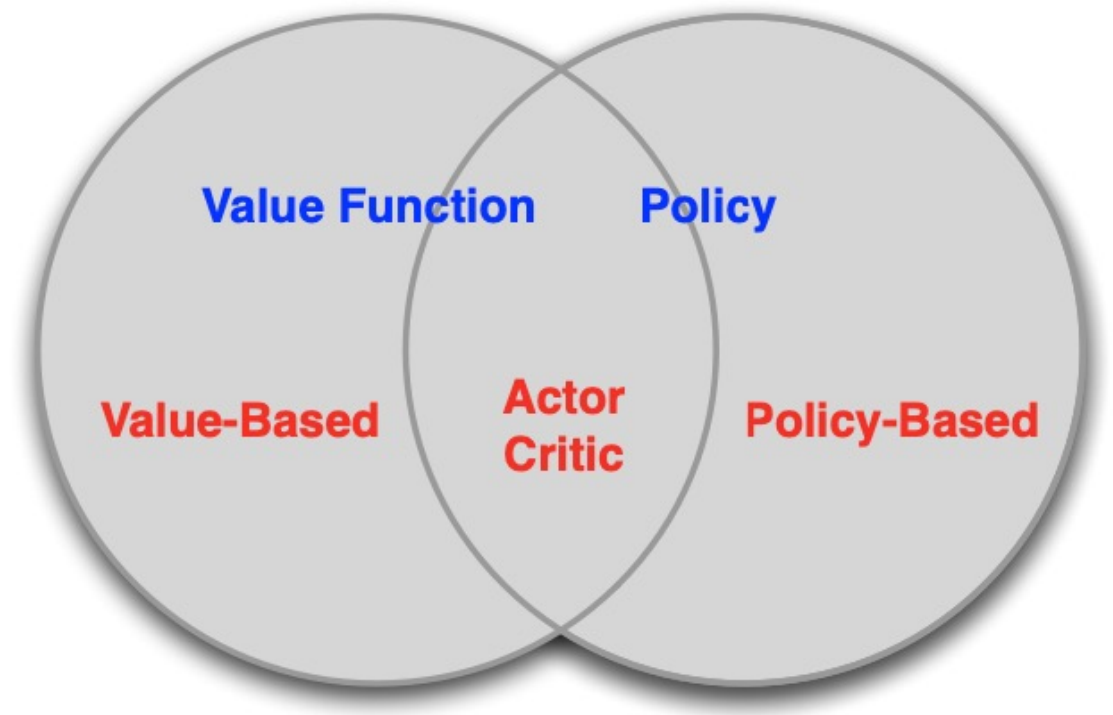
$$\pi_{\theta}(s, a) = \mathbb{P}[a \mid s, \theta]$$

- We return to model-free RL algorithms
(do not need to know the MDP)

Function that gives the probability of selecting action $\mathbf{a}$ in state $\mathbf{s}$, given the policy π , parameterized by set of weights θ

Summary

- **Value-based**
 - Learnt value function
 - Implicit policy (e.g. ϵ -greedy)
- **Policy-based**
 - No value function
 - Learnt policy
- **Actor-critic**
 - Learnt value function
 - Learnt policy



Advantages of Policy-Based RL

- Advantages

- Better convergence properties
- Effective in high-dimensional or continuous action spaces
- Can learn stochastic policies
- Sometime learning a policy is simpler and more efficient

- Disadvantages

- Typically converge to a local rather than global optimum
- Evaluating a policy is typically inefficient and high variance
- Obtained knowledge can be specific and not generalize well
- Sometimes, learning a model is more efficient

Stochastic policies

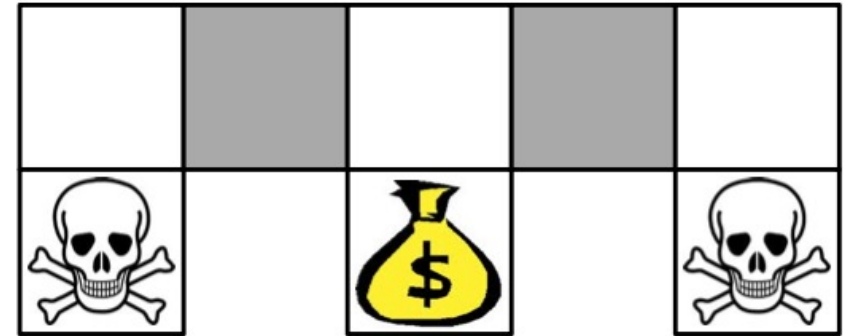
- In MDPs, there is always an optimal deterministic policy
- But, most problems are not fully observable or representable with the “correct” features – common case with function approximation
- In these cases, the optimal policy may be stochastic
- Search space is smoother for stochastic policies – i.e. with gradients
- Provides exploration during learning
- Can be learned by policy gradients

Example – rock, paper, scissors

- Two-player game of rock-paper-scissors
 - Scissors beats paper
 - Rock beats scissors
 - Paper beats rock
- Consider policies for iterated rock-paper-scissors
 - A deterministic policy is easily exploited
 - A uniform random policy is optimal (i.e. Nash equilibrium)



Example – aliased gridworld



- The agent cannot differentiate the grey states (they look the same)
- Consider features of the following form (for all N, E, S, W)

$$\phi(s, a) = \mathbf{1}(\text{wall to N, } a = \text{move E})$$

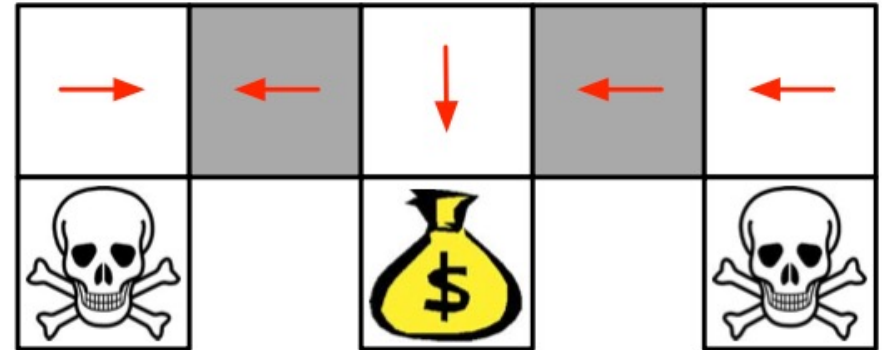
- Compare value-based RL, using an approximate value function

$$Q_{\theta}(s, a) = f(\phi(s, a), \theta)$$

- To policy-based RL, using a parametrised policy

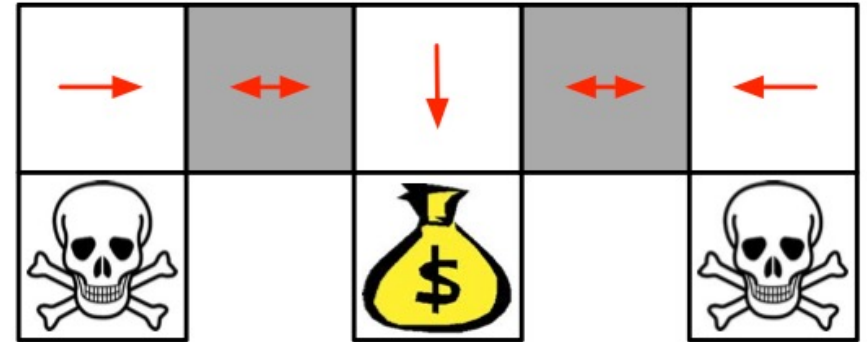
$$\pi_{\theta}(s, a) = g(\phi(s, a), \theta)$$

Example – aliased gridworld



- Under aliasing, an optimal **deterministic policy** will either:
 - move W in both grey states (shown by red arrows)
 - move E in both grey states
- Either way, it can get stuck and never reach the money
- Value-based RL learns a near-deterministic policy (e.g. greedy or ϵ -greedy)
- So it will traverse the corridor for a long time

Example – aliased gridworld



- An optimal **stochastic policy** will randomly move E or W in grey states

$$\pi_{\theta}(\text{wall to N and S, move E}) = 0.5$$

$$\pi_{\theta}(\text{wall to N and S, move W}) = 0.5$$

- It will reach the goal state in a few steps with high probability
- Policy-based RL can learn the optimal stochastic policy

Policy for discrete decisions - softmax

- In the discrete case, when we need to select between a finite set of actions in each state, we will typically use a form of stochastic policy
- We will typically have a preference function $h_{\theta}(s,a)$, which is parameterized by θ and calculated over the selected features - $\phi(s, a)$
- h can be a linear approximator (as we will see later) or any other non linear as a Neural network (e.g. as in the case of AlphaGo)
- The most common stochastic policy function for discrete cases is the **softmax** - probability of action is proportional to exponentiated weight of the preference function for each action

$$\pi(a|s, \theta) \doteq \frac{e^{h(s,a,\theta)}}{\sum_b e^{h(s,b,\theta)}}$$

Policy Objective Functions

- **Goal:** given policy $\pi_\theta(s,a)$ with parameters θ , **find best θ**
- But how do we measure the quality of a policy π_θ ?
- In episodic environments, we can use the **average episode return**

$$\begin{aligned} J_G(\theta) &= \mathbb{E}_{S_0 \sim d_0, \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] \\ &= \mathbb{E}_{S_0 \sim d_0, \pi_\theta} [G_0] \\ &= \mathbb{E}_{S_0 \sim d_0} [\mathbb{E}_{\pi_\theta} [G_t \mid S_t = S_0]] \\ &= \mathbb{E}_{S_0 \sim d_0} [v_{\pi_\theta}(S_0)] \end{aligned}$$

where d_0 is the start-state distribution

Policy Objective Functions

In **continuing environments**, we can use the **average reward** per time step

$$\begin{aligned} J_R(\theta) &= \mathbb{E}_{\pi_\theta} [R_{t+1}] \\ &= \mathbb{E}_{S_t \sim d_{\pi_\theta}} \left[\mathbb{E}_{A_t \sim \pi_\theta(S_t)} [R_{t+1} \mid S_t] \right] \\ &= \sum_s d_{\pi_\theta}(s) \sum_a \pi_\theta(s, a) \sum_r p(r \mid s, a) r \end{aligned}$$

where $d_\pi(s) = p(S_t = s \mid \pi)$ is the probability of being in state s in the long run
Think of it as the ratio of time spent in s under policy π

Policy optimization (gradient)

- Policy based RL is an **optimisation problem**
- Find θ that maximises $J(\theta)$
- Some approaches do not use gradient
 - Hill climbing
 - Simplex / amoeba / Nelder Mead
 - Genetic algorithms
- Greater efficiency often possible using **gradient**
 - Gradient descent
 - Conjugate gradient
 - Quasi-newton
- We focus on **gradient descent**, many extensions possible
- And on methods that exploit sequential structure

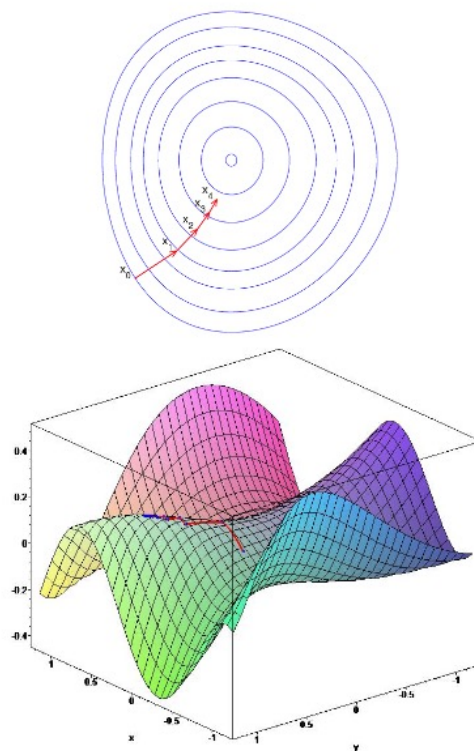
Policy gradient ascent

- Let $J(\theta)$ be any policy objective function
- Policy gradient algorithms search for a **local maximum** in $J(\theta)$ by **ascending** the gradient of the policy w.r.t. parameters θ

$$\Delta\theta = \alpha \nabla_{\theta} J(\theta)$$

- Where $\nabla_{\theta} J(\theta)$ is the policy gradient

$$\nabla_{\theta} J(\theta) = \begin{pmatrix} \frac{\partial J(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{pmatrix}$$



Score function

- We compute the policy gradient **analytically**
- Assume policy π_θ is differentiable whenever it is non-zero and we know the gradient $\nabla_\theta \pi_\theta(s, a)$
- **Likelihood ratios** “trick” exploit the following identity

$$\begin{aligned}\nabla_\theta \pi_\theta(s, a) &= \pi_\theta(s, a) \frac{\nabla_\theta \pi_\theta(s, a)}{\pi_\theta(s, a)} \\ &= \pi_\theta(s, a) \nabla_\theta \log \pi_\theta(s, a)\end{aligned}$$

- And the **score function** is: $\nabla_\theta \log \pi_\theta(s, a)$

Expressions in the pink areas are the same by the rules of calculating derivatives of the log function

Gradient for one-step MDP

- Consider a simple class of one-step MDPs (**contextual bandit**)
 - Starting in state $s \sim d(s)$
 - Terminating after one time-step with reward $r = R_{s,a}$
- Use likelihood ratios to compute the policy gradient

$$J(\theta) = \mathbb{E}_{\pi_\theta} [r]$$

$$= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi_\theta(s, a) \mathcal{R}_{s,a}$$

$$\nabla_\theta J(\theta) = \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi_\theta(s, a) \nabla_\theta \log \pi_\theta(s, a) \mathcal{R}_{s,a}$$

$$= \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(s, a) r]$$

So the update is: $\theta_{t+1} = \theta_t + \alpha R_{t+1} \nabla_\theta \log \pi_{\theta_t}(A_t | S_t)$

Intuition: increase probability of actions with high reward

Score function for Softmax policy

- We will use a softmax policy as a running example

$$\pi_{\theta}(a|s) = \frac{e^{h(s,a)}}{\sum_b e^{h(s,b)}}$$

- The gradient of the log probability

$$\nabla_{\theta} \log \pi_{\theta}(A_t|S_t) = \underbrace{\nabla_{\theta} h(S_t, A_t)}_{\text{gradient of preference}} - \underbrace{\sum_a \pi_{\theta}(a|S_t) \nabla_{\theta} h(S_t, a)}_{\text{expected gradient of preference}}$$

Score function for Softmax policy (linear)

- Weight actions using linear combination of features $\phi(s,a)^\top \theta$
- Probability of action is proportional to exponentiated weight

$$\pi_\theta(s, a) \propto e^{\phi(s,a)^\top \theta}$$

- The score function is

$$\nabla_\theta \log \pi_\theta(s, a) = \phi(s, a) - \mathbb{E}_{\pi_\theta} [\phi(s, \cdot)]$$

Estimated by the mean



Policy gradient theorem (general)

- The policy gradient theorem generalises the likelihood ratio approach to **multi-step MDPs**
- Replaces instantaneous reward r with **long-term value** $Q^\pi(s,a)$ or G_t
- Policy gradient theorem applies to start state objective, average reward and average value objective (with some changes)

Policy gradient theorem - episodic

For any differentiable policy $\pi_{\theta}(s, a)$, let d_0 be the starting distribution over states in which we begin an episode. Then, the policy gradient of $J(\theta) = \mathbb{E}[G_0 \mid S_0 \sim d_0]$ is

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \gamma^t q_{\pi_{\theta}}(S_t, A_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \mid S_0 \sim d_0 \right]$$

where

$$\begin{aligned} q_{\pi}(s, a) &= \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \end{aligned}$$

Here, the objective is **average return per episode**

Episodic policy gradients algorithm

$$\nabla_{\theta} J_{\theta}(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t q_{\pi}(S_t, A_t) \nabla_{\theta} \log \pi(A_t | S_t) \right]$$

- We can sample this, given whole episodes
- We can pull out the sum, and split this into separate gradients

$$\Delta \theta_t = \gamma^t G_t \nabla_{\theta} \log \pi(A_t | S_t)$$

such that $\mathbb{E}_{\pi} [\sum_t \Delta \theta_t] = \nabla_{\theta} J_{\theta}(\pi)$

ignoring the discount $\Delta \theta_t = G_t \nabla_{\theta} \log \pi(A_t | S_t)$

Policy gradient theorem – continuing

For any differentiable policy $\pi_{\theta}(s, a)$, the policy gradient of $J(\theta) = \mathbb{E}[R \mid \pi]$ is

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi}[q_{\pi_{\theta}}(S_t, A_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)]$$

where

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[R_{t+1} - \rho + q_{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

$$\rho = \mathbb{E}_{\pi}[R_{t+1}] \quad (\text{Note: global average, not conditioned on state or action})$$

(Expectation is over both states and actions)

$$\text{Equivalent: } \nabla_{\theta} J(\theta) = \mathbb{E}_{\pi}[R_{t+1} \sum_{n=0}^{\infty} \underbrace{\nabla_{\theta} \log \pi_{\theta}(A_{t-n} | S_{t-n})}_{\mathbb{I}}]$$

Here, the objective is
average rewards

Monte Carlo policy gradient - REINFORCE

- Update parameters by stochastic gradient ascent, using policy gradient theorem
- Using return v_t as an unbiased sample of $Q^{\pi_\theta}(s_t, a_t)$

$$\Delta\theta_t = \alpha \nabla_{\theta} \log \pi_{\theta}(s_t, a_t) v_t$$

function REINFORCE

Initialise θ arbitrarily

for each episode $\{s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T\} \sim \pi_{\theta}$ **do**

for $t = 1$ to $T - 1$ **do**

$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(s_t, a_t) v_t$

end for

end for

return θ

end function

Pseudo-code for REINFORCE

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Algorithm parameter: step size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

 Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \theta)$$

Pseudo-code

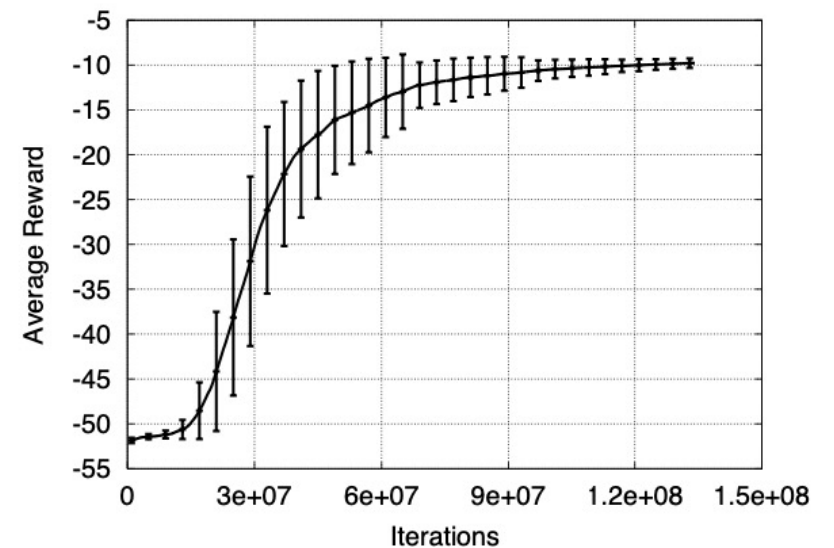
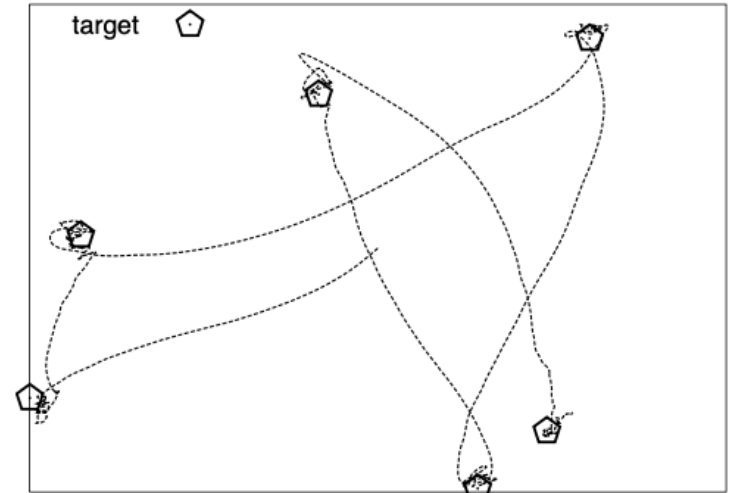
Discounted version

V_t is equal to the return G

$\ln = \log$

Example – puck world

- Continuous actions exert small force on puck
- Puck is rewarded for getting close to target
- Target location is reset every 30 seconds
- Policy is trained using variant of MC policy gradient



Reducing variance using a baseline

- We subtract a baseline function $B(s)$ from the policy gradient
- This can reduce variance, without changing expectation

$$\begin{aligned}\mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) B(s)] &= \sum_{s \in \mathcal{S}} d^{\pi_{\theta}}(s) \sum_a \nabla_{\theta} \pi_{\theta}(s, a) B(s) \\ &= \sum_{s \in \mathcal{S}} d^{\pi_{\theta}}(s) B(s) \nabla_{\theta} \sum_{a \in \mathcal{A}} \pi_{\theta}(s, a) \\ &= 0\end{aligned}$$

- A good baseline is the state value function $B(s) = V^{\pi_{\theta}}(s)$
- So, we can rewrite the policy gradient using the **advantage function**:

$$\begin{aligned}A^{\pi_{\theta}}(s, a) &= Q^{\pi_{\theta}}(s, a) - V^{\pi_{\theta}}(s) \\ \nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) A^{\pi_{\theta}}(s, a)]\end{aligned}$$

Pseudo-code for REINFORCE with baseline

REINFORCE with Baseline (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Algorithm parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

 Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

$$\delta \leftarrow G - \hat{v}(S_t, \mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t, \mathbf{w})$$

$$\theta \leftarrow \theta + \alpha^{\theta} \gamma^t \delta \nabla \ln \pi(A_t|S_t, \theta)$$

Pseudo-code

Value function estimated by MC – weights $\mathbf{w}$

Reducing Variance Using a Critic

- Monte-Carlo policy gradient still has high variance
- We use a **critic** to estimate the action-value function $Q_w(s, a) \approx Q^{\pi_\theta}(s, a)$
- Actor-critic algorithms maintain two sets of parameters:
 - **Critic** - Updates action-value function parameters $\mathbf{w}$
 - **Actor** - Updates policy parameters θ , in direction suggested by critic
- Actor-critic algorithms follow an approximate policy gradient

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(s, a) Q_w(s, a)]$$
$$\Delta\theta = \alpha \nabla_\theta \log \pi_\theta(s, a) Q_w(s, a)$$

Estimating the action-value function

- The critic is solving a familiar problem: **policy evaluation**
- How good is policy π_θ for current parameters θ ?
- This problem was explored in previous classes, e.g.
 - Monte-Carlo policy evaluation
 - Temporal-Difference learning
 - TD(λ)
 - Least-squares

Action-value Actor-Critic

- Simple actor-critic algorithm based on action-value critic – one-step
 - **Critic** Updates w by linear TD(0)
 - **Actor** Updates θ by policy gradient

```
function ONE-STEP ACTOR CRITIC
```

```
  Initialise  $s, \theta$ 
```

```
  for  $t = 0, 1, 2, \dots$  do
```

```
    Sample  $A_t \sim \pi_{\theta}(S_t)$ 
```

```
    Sample  $R_{t+1}$  and  $S_{t+1}$ 
```

```
     $\delta_t = R_{t+1} + \gamma v_w(S_{t+1}) - v_w(S_t)$  [one-step TD-error, or advantage]
```

```
     $w \leftarrow w + \beta \delta_t \nabla_w v_w(S_t)$  [TD(0)]
```

```
     $\theta \leftarrow \theta + \alpha \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$  [Policy gradient update (ignoring  $\gamma^t$  term)]
```

This algorithm can be updated to consider TD(λ) with eligibility traces (check section 13.5 of Sutton& Barto)

Estimating the Advantage Function

- The advantage function can significantly reduce variance of policy gradient
- So, the critic should really estimate the advantage function
- For instance, by estimating both V and Q
- Using two function approximators and two parameter vectors

$$V_v(s) \approx V^{\pi_\theta}(s)$$

$$Q_w(s, a) \approx Q^{\pi_\theta}(s, a)$$

$$A(s, a) = Q_w(s, a) - V_v(s)$$

- And updating both value functions by e.g. TD learning

Increasing robustness with trust regions

- Bad policies lead to bad data !
- One way to increase stability is to regularise
- Popular method: to limit the difference between subsequent policies
- Use the Kullbeck-Leibler divergence:

$$\text{KL}(\pi_{\text{old}} \parallel \pi_{\theta}) = \mathbb{E} \left[\int \pi_{\text{old}}(a \mid S) \log \frac{\pi_{\theta}(a \mid S)}{\pi_{\text{old}}(a \mid S)} \mathrm{d}a \right]$$

- Maximize $J(\theta) - \eta \text{KL}(\pi_{\text{old}} \parallel \pi_{\theta})$
- Examples: TRPO, PPO and MPO algorithms

Score function for Gaussian policy

- In **continuous** action spaces, a Gaussian policy is natural
- Mean is a function of the state $\mu_{\theta}(s)$
- Variance may be fixed σ^2 , or can also be parametrised
- Policy is Gaussian, $A_t \sim N(\mu_{\theta}(S_t), \sigma^2)$
- The score function is:

$$\nabla_{\theta} \log \pi_{\theta}(s, a) = \frac{A_t - \mu_{\theta}(S_t)}{\sigma^2} \nabla \mu_{\theta}(s)$$

- This can be plugged into REINFORCE or actor/critic

Policy gradient with Gaussian policy

- Update:

$$\begin{aligned}\theta_{t+1} &= \theta_t + \beta(G_t - v(S_t))\nabla_{\theta} \log \pi_{\theta}(A_t|S_t) \\ &= \theta_t + \beta(G_t - v(S_t))\frac{A_t - \mu_{\theta}(S_t)}{\sigma^2}\nabla\mu_{\theta}(S_t)\end{aligned}$$

- Intuition: if return is high move $\mu_{\theta}(S_t)$ towards A_t